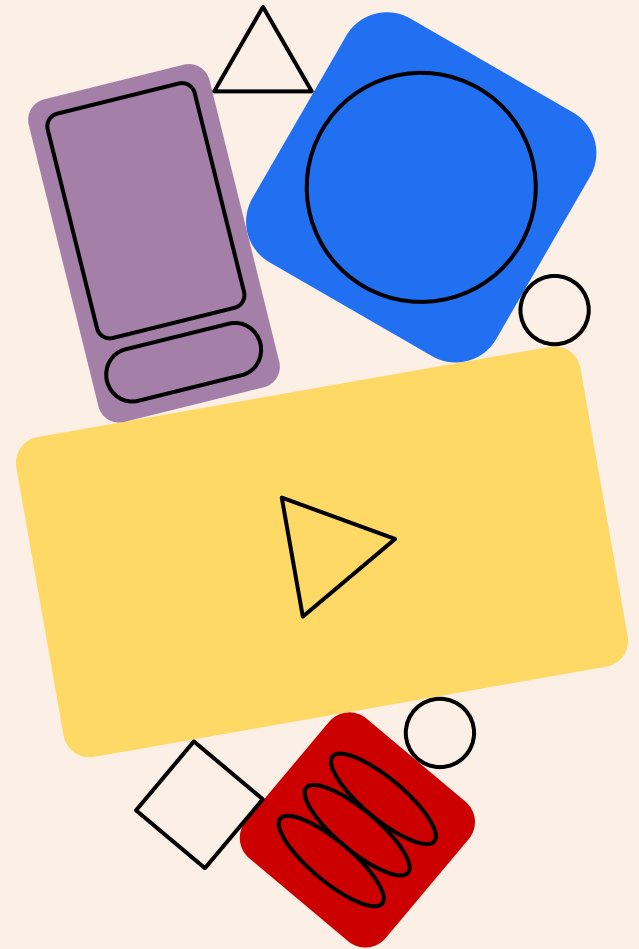


Non Linear Data Structures - Graph



Outline

- Definition of Graph
- Representation of Graphs
- Types of Graph
- Graph Traversal [Depth First Search, Breadth First Search]
- Graph Traversal and Spanning Forest
- Minimum Spanning Tree [Prim's Algorithm and Kruskal's Algorithm]
- Finding the Shortest Path [Dijkstra's Technique]

Introduction

- A graph is a way of representing relationships that exist between pairs of objects.
- That is, a graph is a set of objects, called vertices, together with a collection of pairwise connections between them, called edges.
- Graphs have applications in modeling many domains, including mapping, transportation, computer networks, and electrical engineering.
- Viewed abstractly, a graph **G** is simply a set **V** of vertices and a collection **E** of pairs of vertices from **V**, called edges.
- Thus, a graph is a way of representing connections or relationships between pairs of objects from some set **V**.
- Incidentally, some books use different terminology for graphs and refer to what we call vertices as nodes and what we call edges as arcs. We use the terms “vertices” and “edges.”

Introduction

- Edges in a graph are either **directed** or **undirected**.
- An edge (u,v) is said to be directed from u to v if the pair (u,v) is ordered, with u preceding v .
- An edge (u,v) is said to be undirected if the pair (u,v) is not ordered. Undirected edges are sometimes denoted with set notation, as $\{u,v\}$, but for simplicity we use the pair notation (u,v) , noting that in the undirected case (u,v) is the same as (v,u) .
- Graphs are typically visualized by drawing the vertices as ovals or rectangles and the edges as segments or curves connecting pairs of ovals and rectangles.

Introduction

- A graph is an abstract data structure that is used to implement the mathematical concept of graphs.
- It is basically a collection of vertices (also called nodes) and edges that connect these vertices.
- A graph is often viewed as a generalization of the tree structure, where instead of having a purely parent-to-child relationship between tree nodes, any kind of complex relationship can exist.
- Graphs are widely used to model any situation where entities or things are related to each other in pairs.
- For example, the following information can be represented by graphs:
 - Family trees in which the member nodes have an edge from parent to each of their children.
 - Transportation networks in which nodes are airports, intersections, ports, etc. The edges can be airline flights, one-way roads, shipping routes, etc.

Introduction

- A graph $G = (V(G), E(G))$ consist of two finite sets :
- A graph is a nonempty set of nodes (vertices) and a set of arcs (edges) such that each arc connects two nodes. Here $V(G)$ represents vertices set and $E(G)$ represents edge.

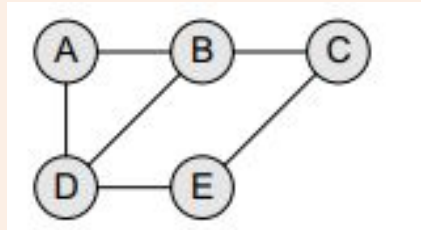
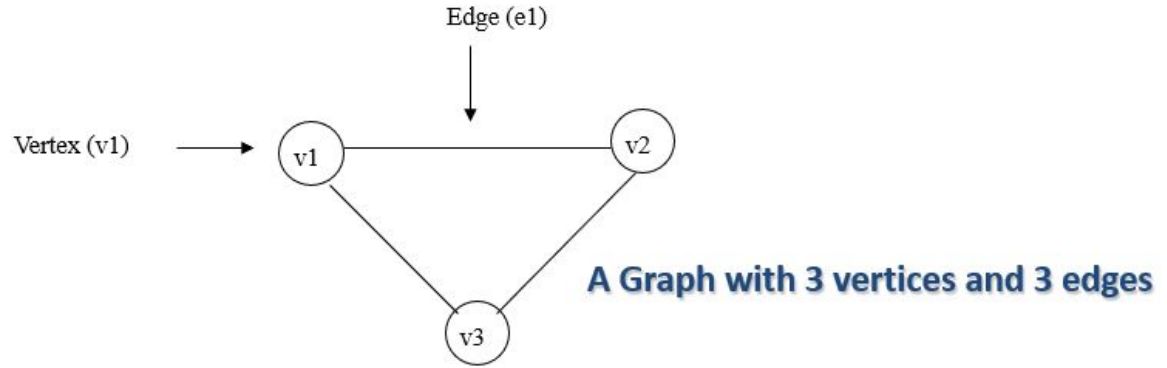
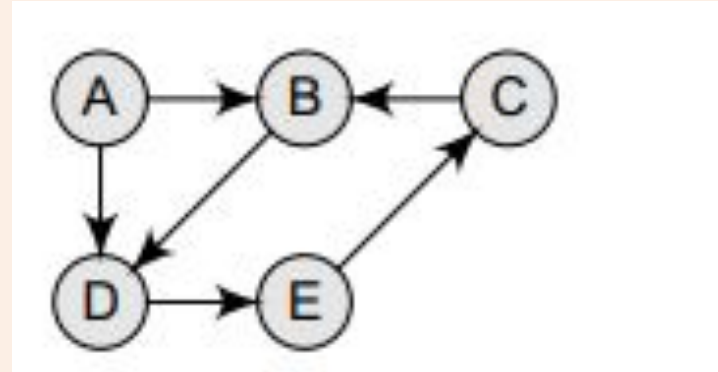
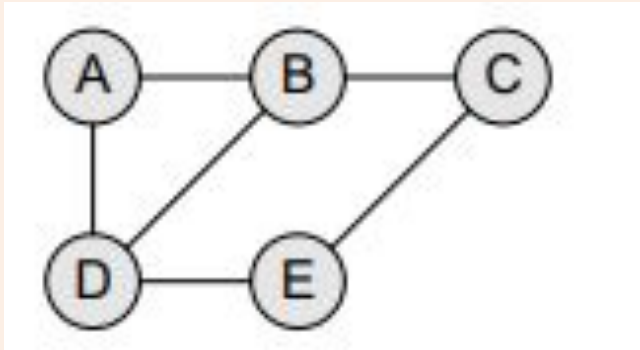


Figure shows a graph with $V(G) = \{A, B, C, D \text{ and } E\}$ and $E(G) = \{(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)\}$.
Note that there are five vertices or nodes and six edges in the graph.

Introduction

- A graph can be directed or undirected.
- In an undirected graph, edges do not have any direction associated with them. That is, if an edge is drawn between nodes A and B, then the nodes can be traversed from A to B as well as from B to A.
- In a directed graph, edges form an ordered pair. If there is an edge from A to B, then there is a path from A to B but not from B to A.
- The edge (A, B) is said to initiate from node A (also known as initial node) and terminate at node B (terminal node).



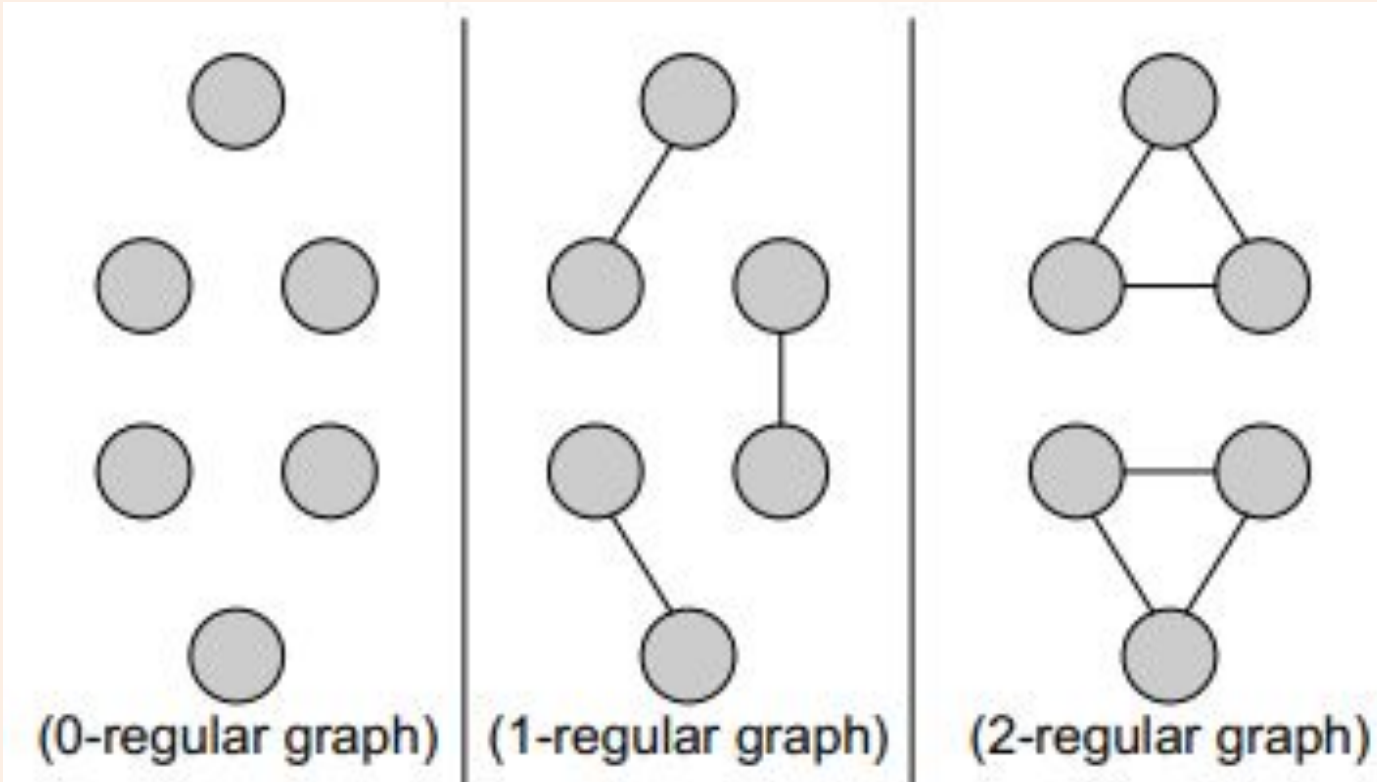
Introduction

- If all the edges in a graph are undirected, then we say the graph is an undirected graph.
- Likewise, a directed graph, also called a digraph, is a graph whose edges are all directed.
- A graph that has both directed and undirected edges is often called a mixed graph.
- Note that an undirected or mixed graph can be converted into a directed graph by replacing every undirected edge (u,v) by the pair of directed edges (u,v) and (v,u) .
- It is often useful, however, to keep undirected and mixed graphs represented as they are, for such graphs have several applications.

Introduction - Terminology

- Adjacent nodes or neighbours
 - For every edge, $e = (u, v)$ that connects nodes u and v , the nodes u and v are the end-points and are said to be the adjacent nodes or neighbours.
- Degree of a node
 - Degree of a node u , $\deg(u)$, is the total number of edges containing the node u . If $\deg(u) = 0$, it means that u does not belong to any edge and such a node is known as an isolated node
- Regular graph
 - It is a graph where each vertex has the same number of neighbours. That is, every node has the same degree. A regular graph with vertices of degree k is called a k -regular graph or a regular graph of degree k .

Introduction - Terminology

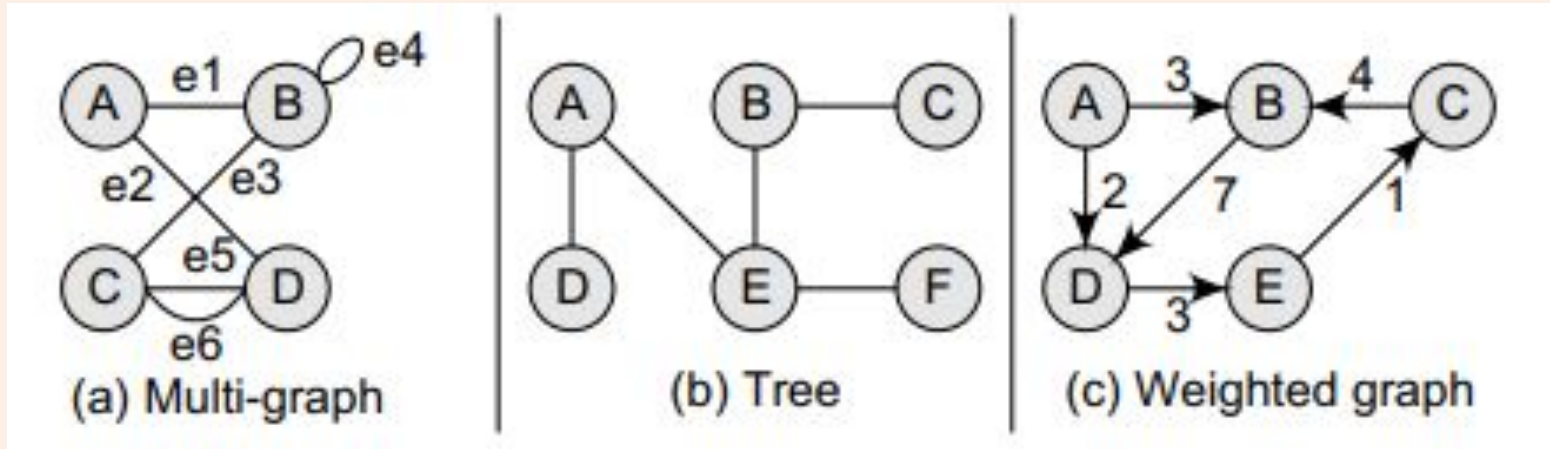


Introduction - Terminology

- Path
 - A path P written as $P = \{v_0, v_1, v_2, \dots, v_n\}$, of length n from a node u to v is defined as a sequence of $(n+1)$ nodes.
 - Here, $u = v_0$, $v = v_n$ and v_{i-1} is adjacent to v_i for $i = 1, 2, 3, \dots, n$.
- Closed path
 - A path P is known as a closed path if the edge has the same end-points. That is, if $v_0 = v_n$.
 - Simple path A path P is known as a simple path if all the nodes in the path are distinct with an exception that v_0 may be equal to v_n . If $v_0 = v_n$, then the path is called a closed simple path.
- Cycle
 - A path in which the first and the last vertices are same. A simple cycle has no repeated edges or vertices (except the first and last vertices).

Introduction - Terminology

- Connected graph
 - A graph is said to be connected if for any two vertices (u, v) in V there is a path from u to v .
 - That is to say that there are no isolated nodes in a connected graph. A connected graph that does not have any cycle is called a tree.
 - Therefore, a tree is treated as a special graph.



Introduction - Terminology

- Complete graph
 - A graph G is said to be complete if all its nodes are fully connected.
 - That is, there is a path from one node to every other node in the graph.
 - A complete graph has $n(n-1)/2$ edges, where n is the number of nodes in G .
- Labelled graph or weighted graph
 - A graph is said to be labelled if every edge in the graph is assigned some data.
 - In a weighted graph, the edges of the graph are assigned some weight or length.
 - The weight of an edge denoted by $w(e)$ is a positive value which indicates the cost of traversing the edge.

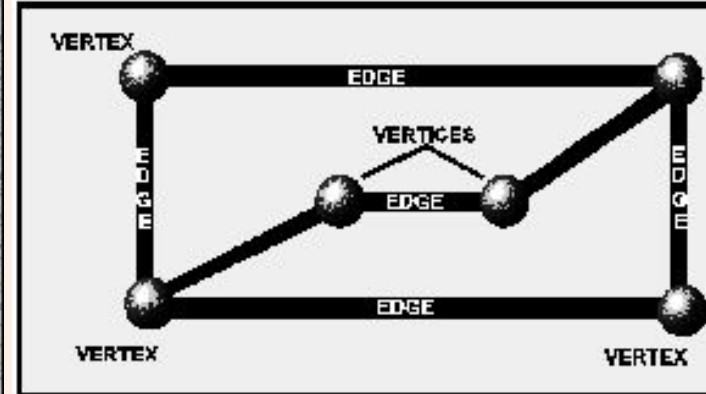
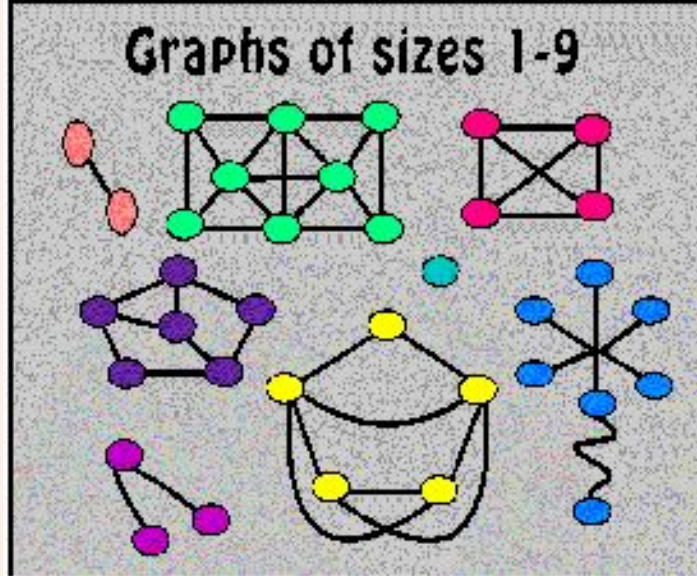
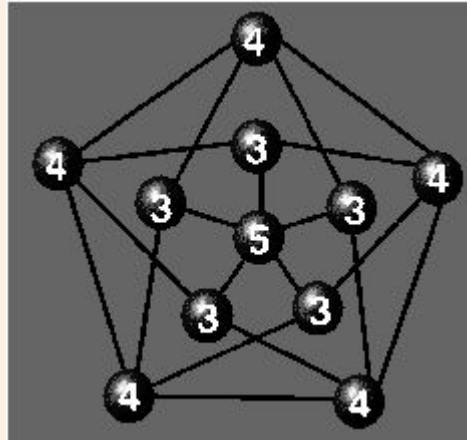
Introduction - Terminology

- Multiple edges
 - Distinct edges which connect the same end-points are called multiple edges. That is, $e = (u, v)$ and $e' = (u, v)$ are known as multiple edges of G .
- Loop
 - An edge that has identical endpoints is called a loop. That is, $e = (u, u)$.
- Multi-graph
 - A graph with multiple edges and/or loops is called a multi-graph.
- Size of a graph
 - The size of a graph is the total number of edges in it.

Refer Book and Resources

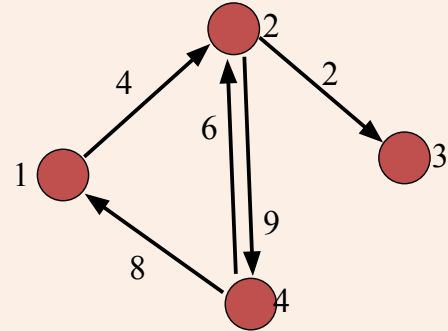
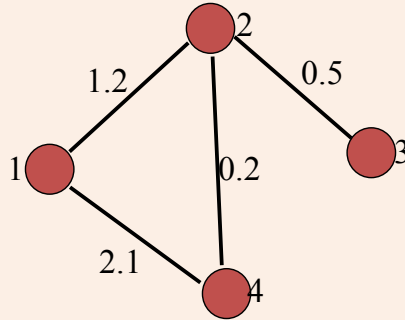
Introduction - Terminology

- Edge – any line drawn from one dot to another
- Vertex – Each dot/point/nodes that appears in a collaboration of dots
- Degree – (of a vertex) The number of edges that touch a vertex
- Size – of a graph is the number of vertices that the graph has



Introduction - Terminology

- A weighted graph is a graph for which each edge has an associated weight, usually given by a weight function
 - $w: E \rightarrow R$
- This could represent distance, energy consumption, cost, etc
- A graph where each edge has a weight is termed an weighted graph
- A graph where edges do not have any associated weights is termed an unweighted graph
- An unweighted graph may be considered to be a weighted graph where all edges have weight 1



Graph ADT

- A graph is a collection of vertices and edges. We model the abstraction as a combination of three data types: Vertex, Edge, and Graph.
- A Vertex is a lightweight object that stores an arbitrary element provided by the user (e.g., an airport code); we assume it supports a method, `element()`, to retrieve the stored element.
- An Edge also stores an associated object (e.g., a flight number, travel distance, cost), retrieved with the `element()` method.
- In addition, we assume that an Edge supports the following methods:

Graph ADT

- In addition, we assume that an Edge supports the following methods:
 - endpoints(): Return a tuple (u,v) such that vertex u is the origin of the edge and vertex v is the destination; for an undirected graph, the orientation is arbitrary.
 - opposite(v): Assuming vertex v is one endpoint of the edge (either origin or destination), return the other endpoint.
- The primary abstraction for a graph is the Graph ADT. We presume that a graph can be either undirected or directed, with the designation declared upon construction; recall that a mixed graph can be represented as a directed graph, modeling edge $\{u,v\}$ as a pair of directed edges (u,v) and (v,u) .

Graph ADT

- The Graph ADT includes the following methods:
 - `vertex count( )`: Return the number of vertices of the graph.
 - `vertices( )`: Return an iteration of all the vertices of the graph.
 - `edge count( )`: Return the number of edges of the graph.
 - `edges( )`: Return an iteration of all the edges of the graph.
 - `get edge(u,v)`: Return the edge from vertex `u` to vertex `v`, if one exists; otherwise return `None`. For an undirected graph, there is no difference between `get edge(u,v)` and `get edge(v,u)`.
 - `degree(v, out=True)`: For an undirected graph, return the number of edges incident to vertex `v`. For a directed graph, return the number of outgoing (resp. incoming) edges incident to vertex `v`, as designated by the optional parameter.

Graph ADT

- The Graph ADT includes the following methods:
 - incident edges(v , out=True): Return an iteration of all edges incident to vertex v . In the case of a directed graph, report outgoing edges by default; report incoming edges if the optional parameter is set to False.
 - insert vertex(x =None): Create and return a new Vertex storing element x .
 - insert edge(u , v , x =None): Create and return a new Edge from vertex u to vertex v , storing element x (None by default).
 - remove vertex(v): Remove vertex v and all its incident edges from the graph.
 - remove edge(e): Remove edge e from the graph.

Representation of Graph

- There are three common ways of storing graphs in the computer's memory. They are:
 - Sequential representation by using an **adjacency matrix**.
 - Linked representation by using an **adjacency list** that stores the neighbours of a node using a linked list.
 - Adjacency multi-list which is an extension of linked representation.
 - Adjacency Map etc.

Representation of Graph

- **Matrix** : With help of Adjacency Matrix.
- **Linked list** : With help of Adjacency List
- Comparing the two representation
 - Space Complexity
 - Adjacency matrix is $O(n^2)$
 - Adjacency list is $O(n+E)$ where E is no. of edges.
 - Static versus dynamic representation
 - An adjacency matrix is static representation : the graph is built in one go and is difficult to alter once built.
 - An adjacency list is a dynamic representation : the graph is built incrementally, thus is more easily altered during run-time

Adjacency Matrix

- An adjacency matrix is used to represent which nodes are adjacent to one another. By definition, two nodes are said to be adjacent if there is an edge connecting them.
- In a directed graph G , if node v is adjacent to node u , then there is definitely an edge from u to v . That is, if v is adjacent to u , we can get from u to v by traversing one edge.
- For any graph G having n nodes, the adjacency matrix will have the dimension of $n \times n$.
- Since an adjacency matrix contains only 0s and 1s, it is called a bit matrix or a Boolean matrix.
- The entries in the matrix depend on the ordering of the nodes in G .
- Therefore, a change in the order of nodes will result in a different adjacency matrix.

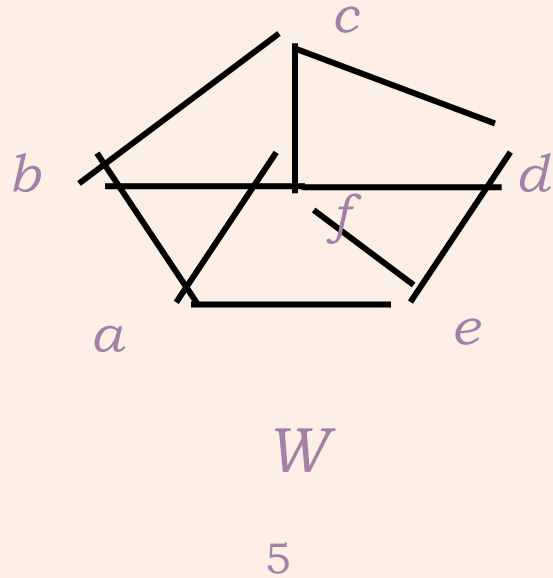
Adjacency Matrix

- A simple graph $G = (V, E)$ with n vertices can be represented by its adjacency matrix, A , where the entry a_{ij} in row i and column j is:

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge in } G \\ 0 & \text{otherwise} \end{cases}$$

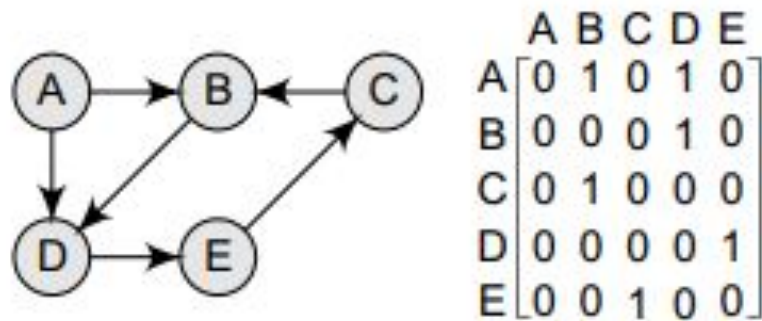
- The adjacency matrix for an undirected graph is symmetric;
- the adjacency matrix for a digraph need not be symmetric

Adjacency Matrix

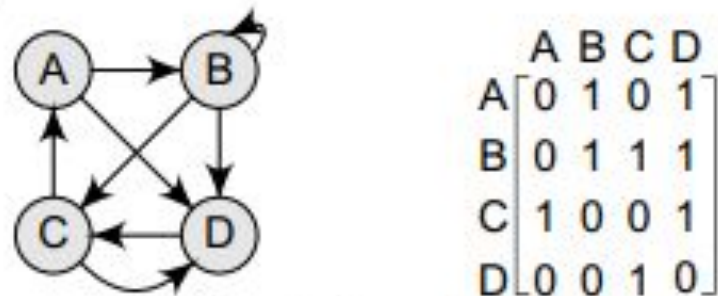


From	To					
	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
<i>a</i>	0	1	0	0	1	1
<i>b</i>	1	0	1	0	0	1
<i>c</i>	0	1	0	1	0	1
<i>d</i>	0	0	1	0	1	1
<i>e</i>	1	0	0	1	0	1
<i>f</i>	1	1	1	1	1	0

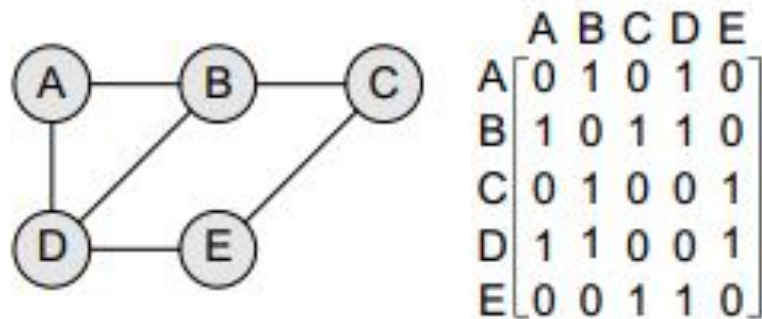
Adjacency Matrix



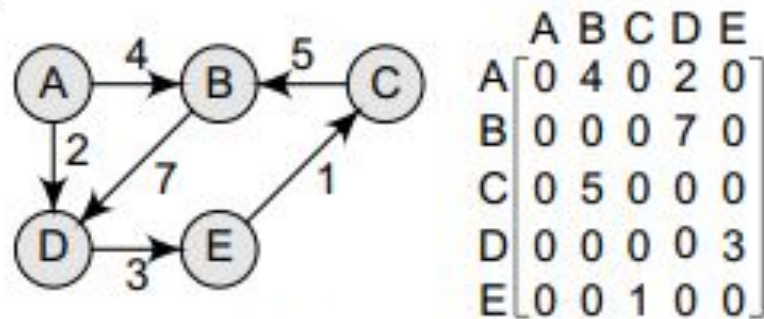
(a) Directed graph



(b) Directed graph with loop



(c) Undirected graph



(d) Weighted graph

Adjacency Matrix

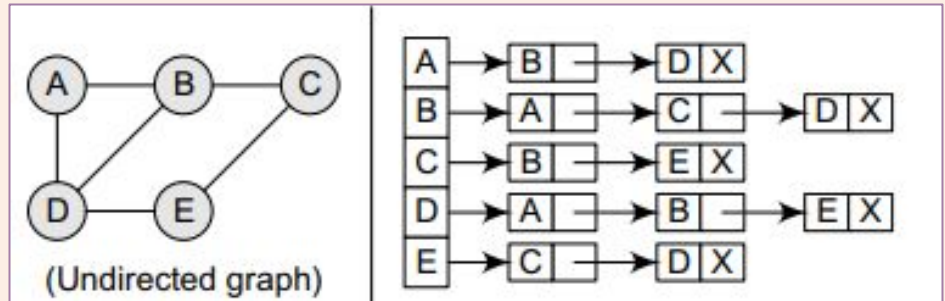
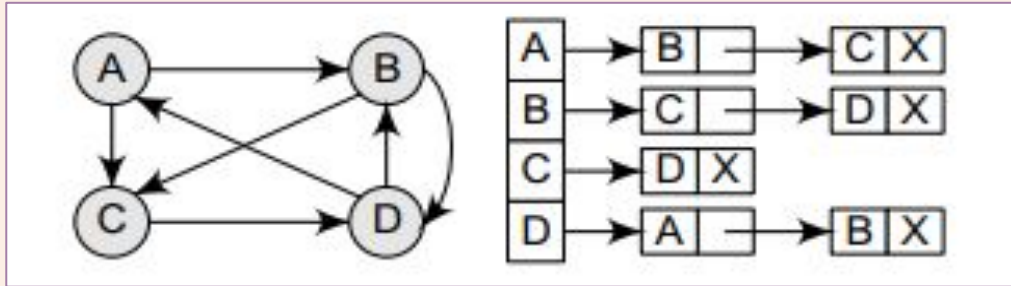
From the above examples, we can draw the following conclusions:

- For a simple graph (that has no loops), the adjacency matrix has 0s on the diagonal.
- The adjacency matrix of an undirected graph is symmetric.
- The memory use of an adjacency matrix is $O(n^2)$, where n is the number of nodes in the graph.
- Number of 1s (or non-zero entries) in an adjacency matrix is equal to the number of edges in the graph.
- The adjacency matrix for a weighted graph contains the weights of the edges connecting the nodes.

Adjacency Matrix - Path Matrix

Adjacency List

- An adjacency list is another way in which graphs can be represented in the computer's memory.
- This structure consists of a list of all nodes in G.
- Furthermore, every node is in turn linked to its own list that contains the names of all other nodes that are adjacent to it.

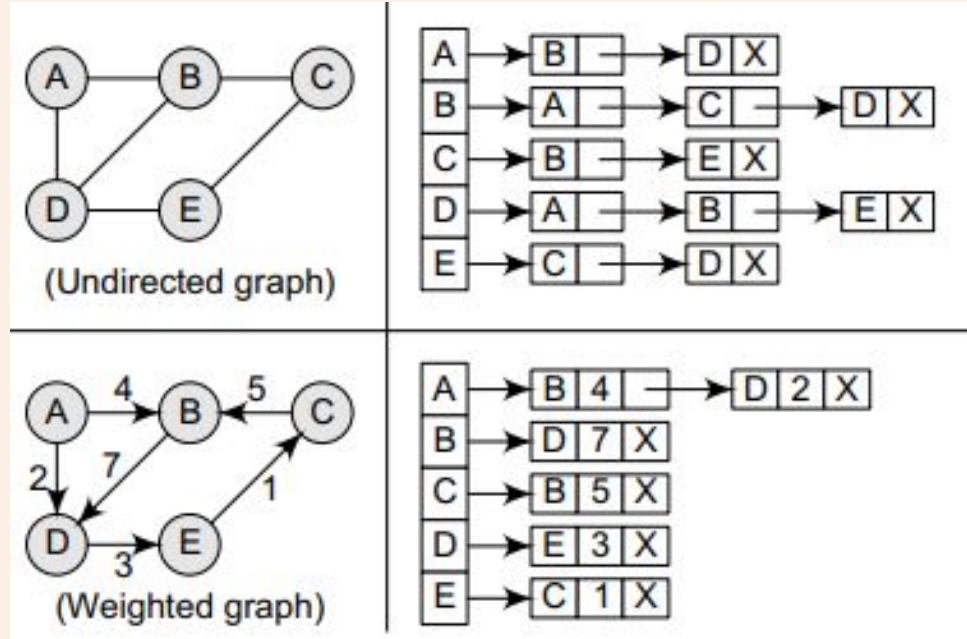


Adjacency List

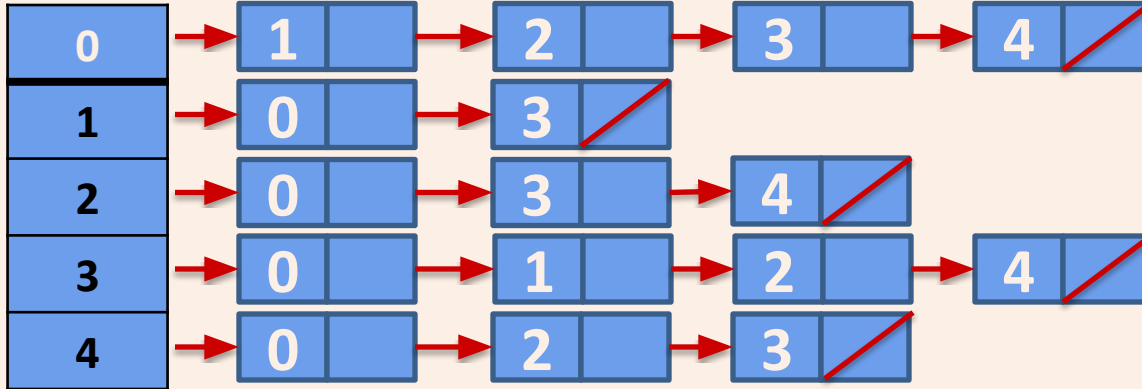
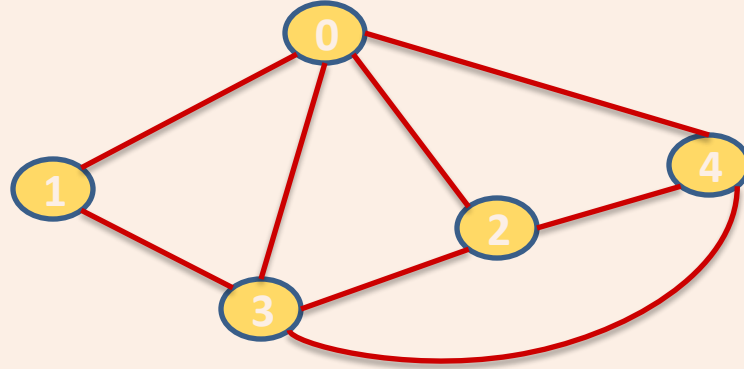
- The key advantages of using an adjacency list are:
- It is easy to follow and clearly shows the adjacent nodes of a particular node.
- It is often used for storing graphs that have a small-to-moderate number of edges. That is, an adjacency list is preferred for representing sparse graphs in the computer's memory; otherwise, an adjacency matrix is a good choice.
- Adding new nodes in G is easy and straightforward when G is represented using an adjacency list. Adding new nodes in an adjacency matrix is a difficult task, as the size of the matrix needs to be changed and existing nodes may have to be reordered.

Adjacency List

- For a directed graph, the sum of the lengths of all adjacency lists is equal to the number of edges in G .
- However, for an undirected graph, the sum of the lengths of all adjacency lists is equal to twice the number of edges in G because an edge (u, v) means an edge from node u to v as well as an edge from v to u .
- Adjacency lists can also be modified to store weighted graphs.



Adjacency List



Traversing a Graph

- Traversing a graph means visiting all the vertices of the graph exactly once.
- It can be started from any vertex of the graph.
- Graph traversing algorithms are :
 - Depth-First Search (DFS)
 - Breadth-First Search (BFS)
- Common Traversal Steps
 - Start from any vertex of a graph
 - From this starting vertex, traverse as deep as you can go.
Whenever you cannot go further, then backtrack one vertex and do the same traversal from this vertex until you cannot traverse further, and so on.
 - Process the information contained in that vertex.
 - Then move along an edge to process a neighbor (adjacent vertex)
 - When the traversal finishes, all the above vertices that can be reached from the start vertex are processed.

Traversing a Graph

- While breadth-first search uses a queue as an auxiliary data structure to store nodes for further processing,
- The depth-first search scheme uses a stack.
- But both these algorithms make use of a variable STATUS.
- During the execution of the algorithm, every node in the graph will have the variable STATUS set to 1 or 2, depending on its current state.

Status	State of the node	Description
1	Ready	The initial state of the node N
2	Waiting	Node N is placed on the queue or stack and waiting to be processed
3	Processed	Node N has been completely processed

Depth-First Search [DFS]

- The depth-first search algorithm progresses by expanding the starting node of G and then going deeper and deeper until the goal node is found, or until a node that has no children is encountered.
- When a dead-end is reached, the algorithm backtracks, returning to the most recent node that has not been completely explored.
- In other words, depth-first search begins at a starting node A which becomes the current node.
- Then, it examines each node N along a path P which begins at A . That is, we process a neighbour of A , then a neighbour of neighbour of A , and so on.
- During the execution of the algorithm, if we reach a path that has a node N that has already been processed, then we backtrack to the current node.
- Otherwise, the unvisited (unprocessed) node becomes the current node.

Depth-First Search [DFS]

- The algorithm proceeds like this until we reach a dead-end (end of path P).
- On reaching the deadend, we backtrack to find another path P'.
- The algorithm terminates when backtracking leads back to the starting node A.
- In this algorithm, edges that lead to a new vertex are called discovery edges and edges that lead to an already visited vertex are called back edges.
- Observe that this algorithm is similar to the in-order traversal of a binary tree.
- Its implementation is similar to that of the breadth-first search algorithm but here we use a stack instead of a queue.
- Again, we use a variable STATUS to represent the current state of the node.

Depth-First Search [DFS]

- Step 1: SET STATUS = 1 (ready state) for each node in G
- Step 2: Push the starting node A on the stack and set its STATUS = 2 (waiting state)
- Step 3: Repeat Steps 4 and 5 until STACK is empty
- Step 4: Pop the top node N. Process it and set its STATUS = 3 (processed state)
- Step 5: Push on the stack all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
- [END OF LOOP]
- Step 6: EXIT

Depth-First Search [DFS]

- Once a possible path is found, continue the search until the end of the path
- DFS follows the following rules:
 - Select an unvisited node x , visit it, and treat as the current node
 - Find an unvisited neighbor of the current node, visit it, and make it the new current node
 - If the current node has no unvisited neighbors, backtrack to its parent, and make that parent the new current node
 - Repeat steps 3 and 4 until no more nodes can be visited.
 - If there are still unvisited nodes, repeat from step 1.

•

Depth-First Search [DFS]

- It is like preorder traversal of tree [Root,Left,Right]
- Traversal can start from any vertex V_i
- V_i is visited and then all vertices adjacent to V_i are traversed recursively using DFS

DFS (G, 1) is given by

Step 1: Visit (1)

Step 2: DFS (G, 2)

DFS (G, 3)

DFS (G, 4)

DFS (G, 5)

→ DFS (G, 2):

Step1: Visit(2)

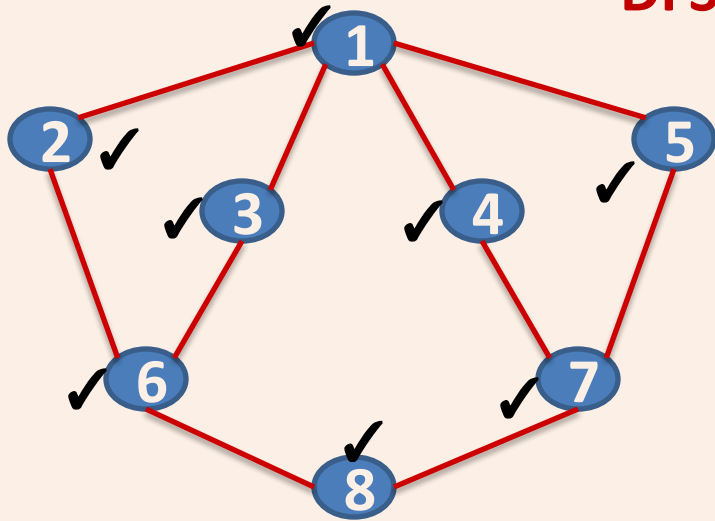
Step 2: DFS (G, 6)

→ DFS (G, 6):

Step1: Visit(6)

Step 2: DFS (G, 3)

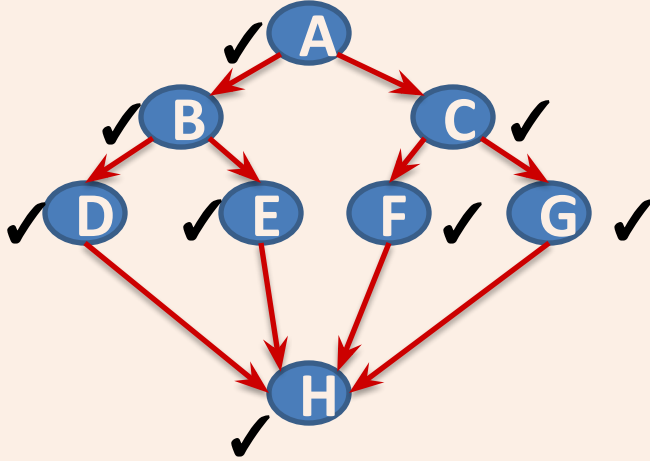
DFS (G, 8)



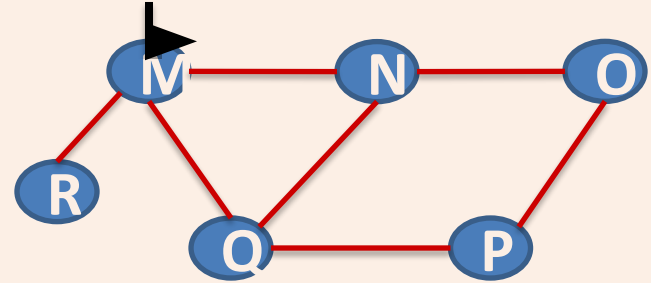
DFS of given graph starting **from** node **1** is given by

1 2 6 3 8 7 4 5

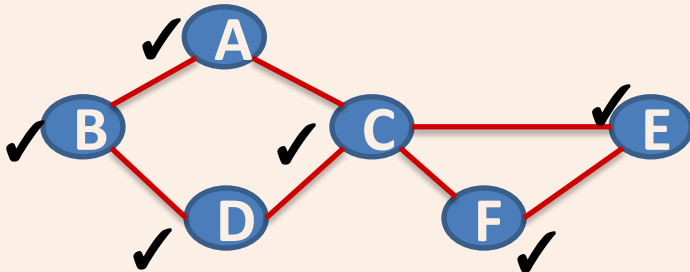
Depth-First Search [DFS]



A B D H E C F G



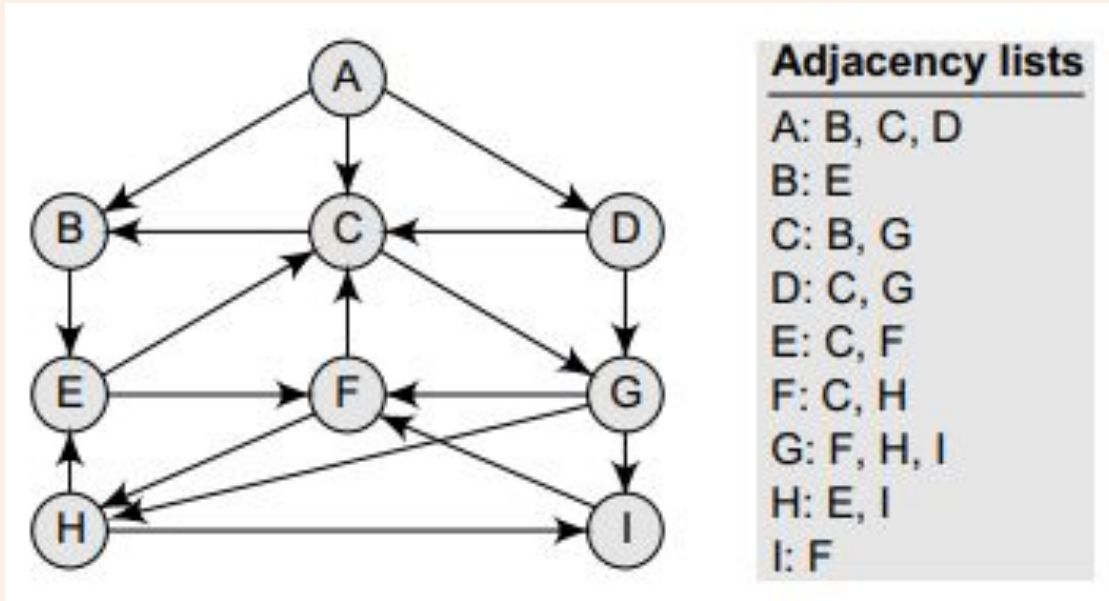
A B D C F E



Depth-First Search [DFS]

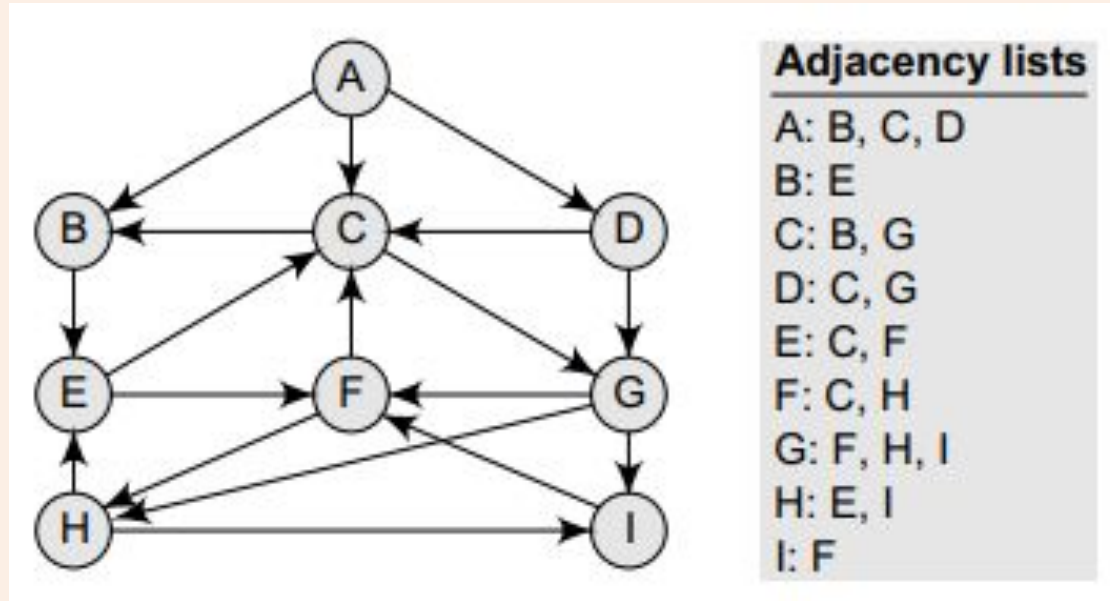
- Features of Depth-First Search Algorithm - **Refer Book and References**
- Depth-first search is useful for:
 - Finding a path between two specified nodes, u and v , of an unweighted graph.
 - Finding a path between two specified nodes, u and v , of a weighted graph.
 - Finding whether a graph is connected or not.
 - Computing the spanning tree of a connected graph.

Depth-First Search [DFS]



Consider the graph G given in Fig. The adjacency list of G is also given. Suppose we want to print all the nodes that can be reached from the node H (including H itself). One alternative is to use a depth-first search of G starting at node H .

Depth-First Search [DFS]



Consider the graph G given in Fig. The adjacency list of G is also given. Suppose we want to print all the nodes that can be reached from the node H (including H itself). One alternative is to use a depth-first search of G starting at node H. >> **H, I, F, C, G, B, E**

Breadth-First Search [BFS]

- Breadth-first search (BFS) is a graph search algorithm that begins at the root node and explores all the neighbouring nodes.
- Then for each of those nearest nodes, the algorithm explores their unexplored neighbour nodes, and so on, until it finds the goal.
- That is, we start examining the node A and then all the neighbours of A are examined.
- In the next step, we examine the neighbours of neighbours of A, so on and so forth.
- This means that we need to track the neighbours of the node and guarantee that every node in the graph is processed and no node is processed more than once.
- This is accomplished by using a queue that will hold the nodes that are waiting for further processing and a variable STATUS to represent the current state of the node.

Breadth-First Search [BFS]

Start several paths at a time, and advance in each one step at a time

BFS follows the following rules:

1. First choose a starting vertex
2. Find all the vertices which are connected to the starting vertex.
3. Then choose one of the connected vertices that are connected to this vertex.
4. Continue this procedure until all the vertices are visited.

Implementation of BFS

- Observations: The first node visited in each level is the first node from which to proceed to visit new nodes.
- This suggests that a queue is the proper data structure to remember the order of the steps.

Breadth-First Search [BFS]

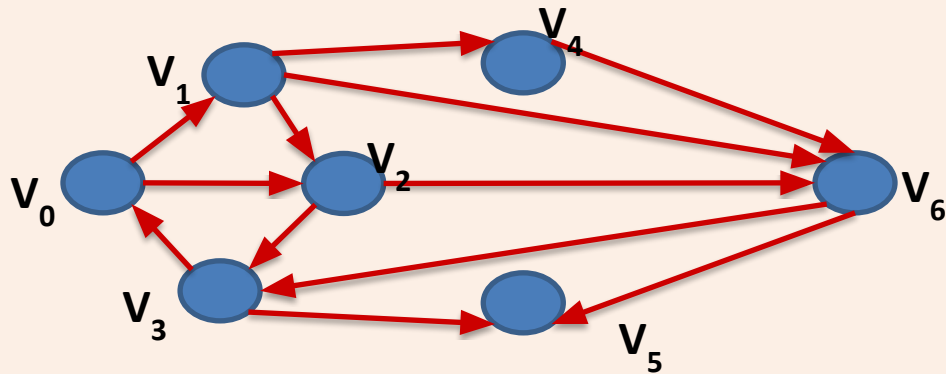
```
Step 1: SET STATUS = 1 (ready state)
        for each node in G
Step 2: Enqueue the starting node A
        and set its STATUS = 2
        (waiting state)
Step 3: Repeat Steps 4 and 5 until
        QUEUE is empty
Step 4: Dequeue a node N. Process it
        and set its STATUS = 3
        (processed state).
Step 5: Enqueue all the neighbours of
        N that are in the ready state
        (whose STATUS = 1) and set
        their STATUS = 2
        (waiting state)
        [END OF LOOP]
Step 6: EXIT
```

Breadth-First Search [BFS]

- This method starts from vertex V_0
- V_0 is marked as visited. All vertices adjacent to V_0 are visited next
- Let vertices adjacent to V_0 are V_1, V_2, V_3, V_4
- V_1, V_2, V_3 and V_4 are marked visited
- All unvisited vertices adjacent to V_1, V_2, V_3, V_4 are visited next
- The method continues until all vertices are visited
- The algorithm for BFS has to maintain a list of vertices which have been visited but not explored for adjacent vertices
- The vertices which have been visited but not explored for adjacent vertices can be stored in queue

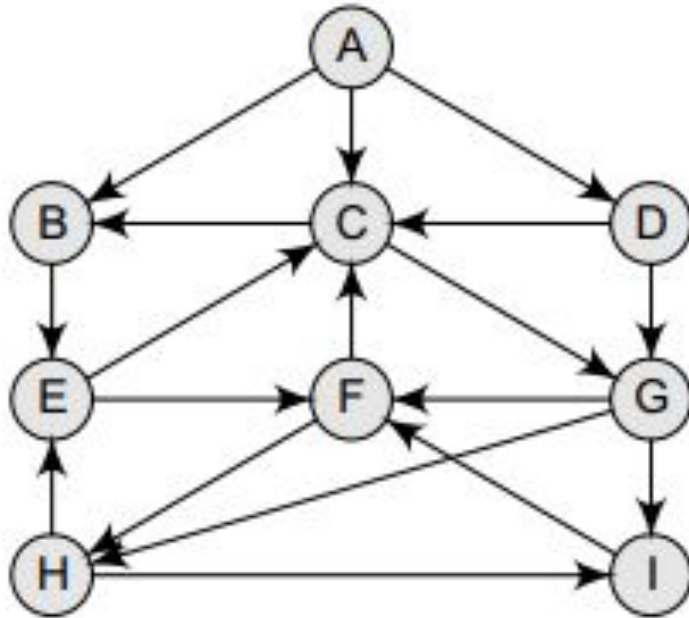
•

Breadth-First Search [BFS]



V_0 | V_1 V_2 | V_4 V_6 V_3 | V_5

Breadth-First Search Program



Adjacency lists

A: B, C, D

B: E

C: B, G

D: C, G

E: C, F

F: C, H

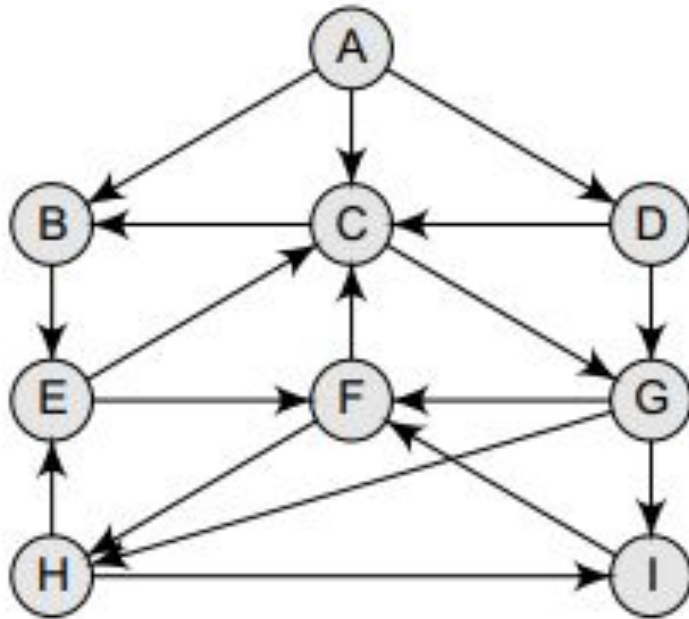
G: F, H, I

H: E, I

I: F

Consider the graph G given in Fig. The adjacency list of G is also given. Assume that G represents the daily flights between different cities and we want to fly from city A to I with minimum stops. That is, find the minimum path P from A to I given that every edge has a length of 1.

Breadth-First Search Program



Adjacency lists

A: B, C, D

B: E

C: B, G

D: C, G

E: C, F

F: C, H

G: F, H, I

H: E, I

I: F

Consider the graph G given in Fig. The adjacency list of G is also given. Assume that G represents the daily flights between different cities and we want to fly from city A to I with minimum stops. That is, find the minimum path P from A to I given that every edge has a length of 1. >> **A -> C -> G -> I.**

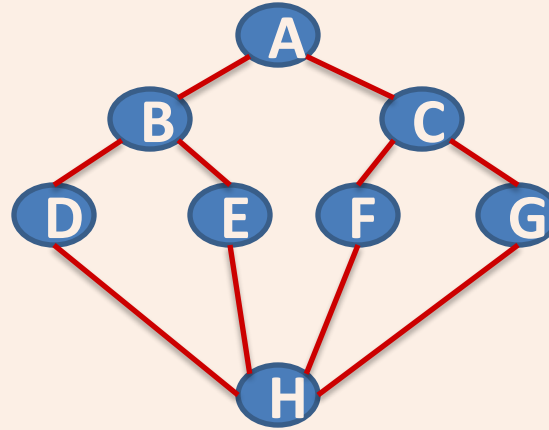
Breadth-First Search [BFS]

- Features of Breadth-First Search Algorithm - **Book and Ref**
- Breadth-first search can be used to solve many problems such as:
 - Finding all connected components in a graph G .
 - Finding all nodes within an individual connected component.
 - Finding the shortest path between two nodes, u and v , of an unweighted graph.
 - Finding the shortest path between two nodes, u and v , of a weighted graph.
 -

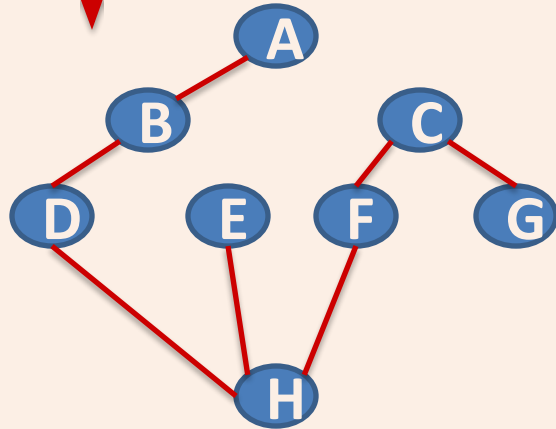
Minimal Spanning Tree(MST)

- **Weighted Spanning Tree:** If graph G is weighted graph, then the weight of a spanning tree T of G is defined as the sum of the weights of all the branches in T then T is called the weighted spanning tree.
- **Minimal Spanning Tree:** A spanning tree with the smallest weight in a weighted graph is called a shortest spanning tree or shortest-distance spanning tree or minimal spanning tree.
- Two techniques for Constructing minimum cost spanning tree
 - Prim's Algorithm
 - Kruskal's Algorithm

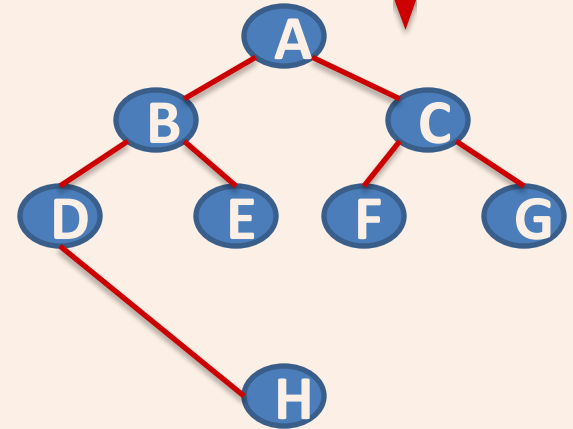
Construct Spanning Tree



**DFS
Spanning
Tree**

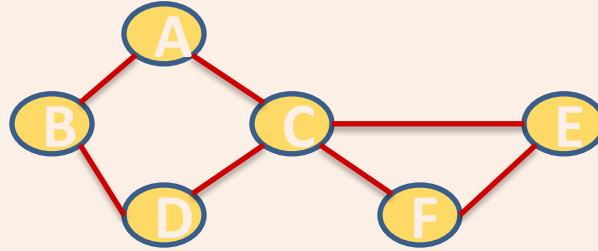
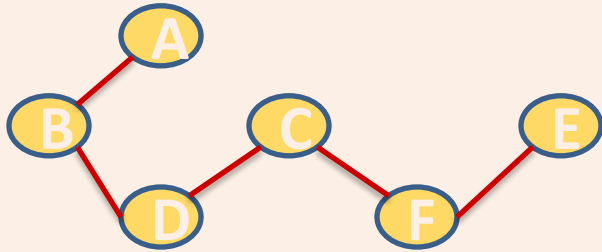


**BFS
Spanning
Tree**

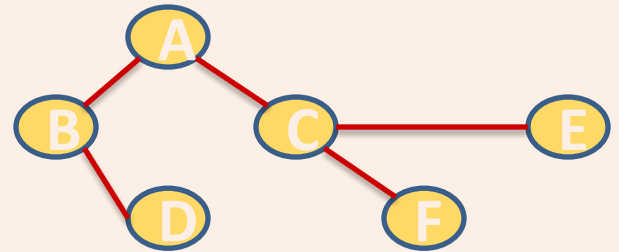


Construct Spanning Tree

**DFS
Spanning
Tree**



**BFS
Spanning
Tree**

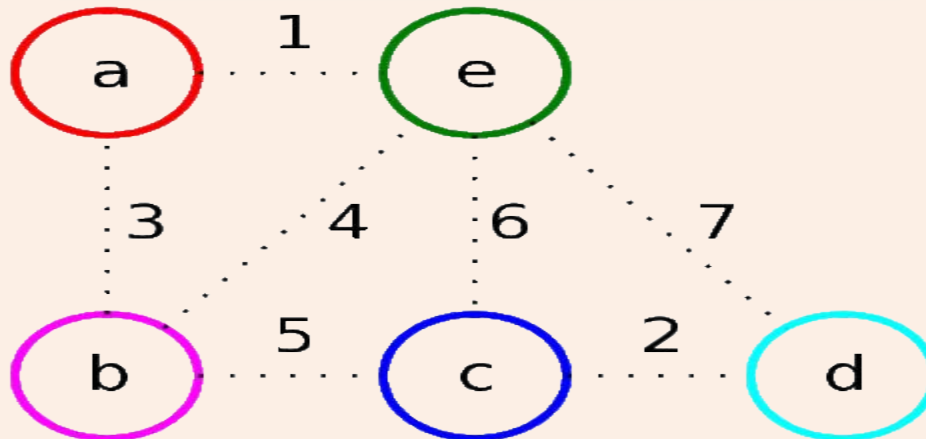


Minimal Spanning Tree Algorithms

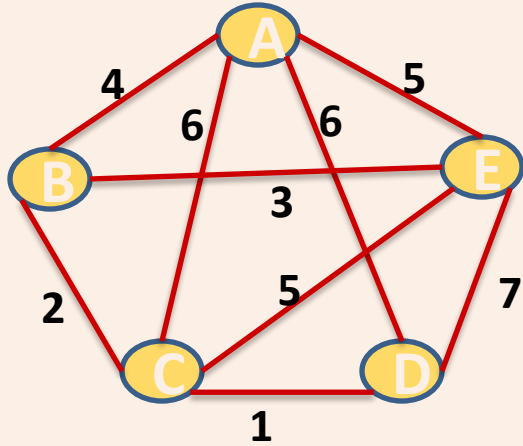
Kruskal's algorithm

1. Select the shortest edge in a network
2. Select the next shortest edge which does not create a cycle
3. Repeat step 2 until all vertices have been connected

Edge	ab	ae	bc	be	cd	ed	ec
Weight	3	1	5	4	2	7	6



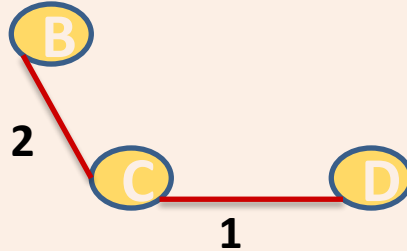
Kruskal's Algorithms



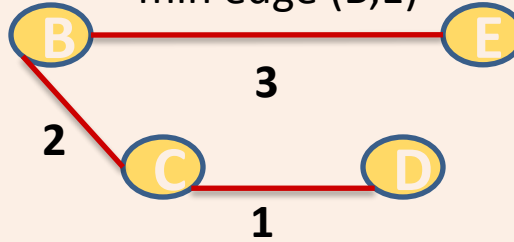
Step 1: Taking min edge (C,D)



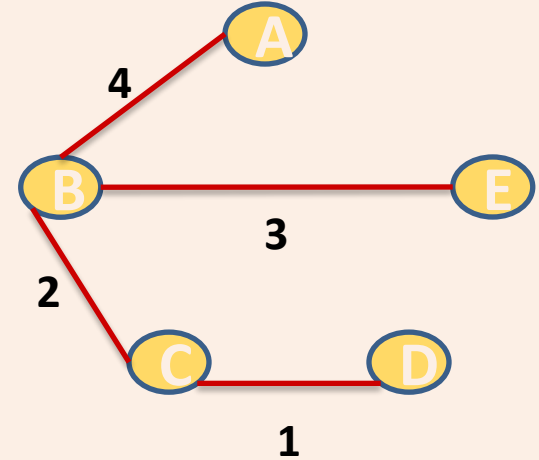
Step 2: Taking next min edge (B,C)



Step 3: Taking next min edge (B,E)



Step 4: Taking next min edge (A,B)



so we obtained minimum spanning tree of cost:

$$4 + 2 + 1 + 3 = 10$$

Implementing Kruskal Algorithms

```
#include<stdio.h>
int i,j,k,a,b,u,v,n,ne=1;
int min,mincost=0,cost[9][9],parent[9];
int find(int);
int uni(int,int);
int main()
{
    printf("\nEnter the no. of vertices:");
    scanf("%d",&n);
    printf("\nEnter the cost adjacency matrix\n");
    for(i=1;i<=n;i++)
        { printf("\n");
          for(j=1;j<=n;j++)
              {
                  printf("a[%d][%d]: ",i,j);
                  scanf("%d",&cost[i][j]);
              }
        }
}
```

Implementing Kruskal Algorithms

```
printf("\nThe edges of Minimum Cost Spanning Tree  
are\n\n");  
while(ne<n) {  
    for(i=1,min=999;i<=n;i++) {  
        for(j=1;j<=n;j++) {  
            if(cost[i][j]<min) {  
                min=cost[i][j];  
                a=u=i;  
                b=v=j;  
            }  
        }  
    }  
    u=find(u);  
    v=find(v);  
    if(uni(u,v)) {  
        printf("\n%d edge (%d,%d) =%d\n",ne++,a,b,min);  
        mincost +=min; }  
}
```

Implementing Kruskal Algorithms

```
int find(int i)
{
    while(parent[i])
        i=parent[i];
    return i;
}
```

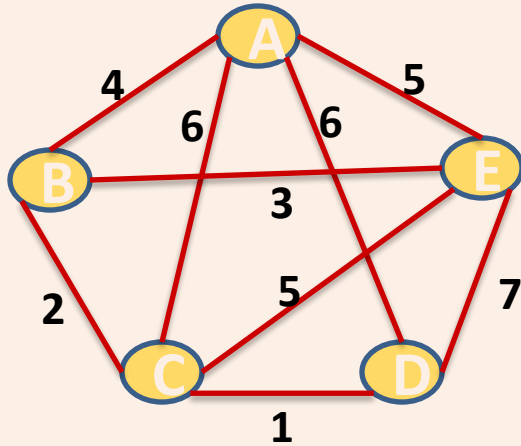
```
int uni(int i,int j)
{
    if(i!=j)
    {
        parent[j]=i;
        return 1;
    }
}
```

Minimal Spanning Tree Algorithms

Prim's algorithm

1. Select any vertex
2. Select the shortest edge connected to that vertex
3. Select the shortest edge connected to any vertex already connected
4. Repeat step 3 until all vertices have been connected

Prims Algorithms



A - B 4	A - D 6	C - E 5
A - E 5	B - E 3	C - D 1
A - C 6	B - C 2	D - E 7

Let X be the set of nodes explored, initially $X = \{A\}$

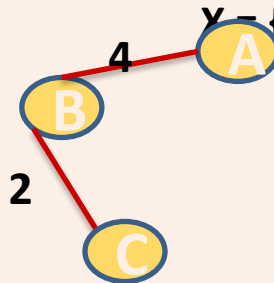


Step 1: Taking minimum Weight edge of all Adjacent edges of $X = \{A\}$



$X = \{A, B\}$

Step 2: Taking minimum weight edge of all Adjacent edges of $Y = \{A, B\}$

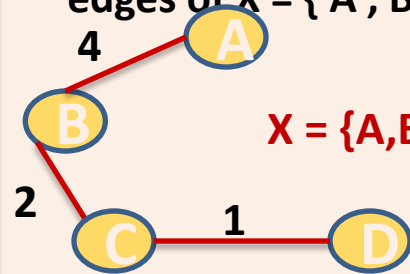


$X = \{A, B, C\}$

We obtained minimum spanning tree of cost:

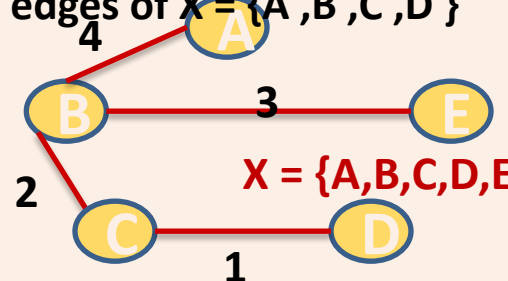
$$4 + 2 + 1 + 3 = 10$$

Step 3: Taking minimum weight edge of all Adjacent edges of $X = \{A, B, C\}$



$X = \{A, B, C, D\}$

Step 4: Taking minimum weight edge of all Adjacent edges of $X = \{A, B, C, D\}$



$X = \{A, B, C, D, E\}$

Implementing Prim's Algorithm

```
# include<stdio.h>
```

```
int G[50][50],select[50], i, j, k, n, min_dist,total=0,u, v;
```

```
/* This function finds the minimal spanning tree by Prim's Algorithm */
```

```
void Prim()
```

```
{
```

```
    printf("\n\n The Minimal Spanning Tree Is :\n");
```

```
    select[0] = 1;
```

```
    for (k=1 ; k<n ; k++) {
```

```
        min_dist = 32767;
```

```
        for (i=0 ; i<n ; i++)
```

Implementing Prim's Algorithm

```
int main()
{
    printf("\n Enter Number of Nodes in The Graph: ");

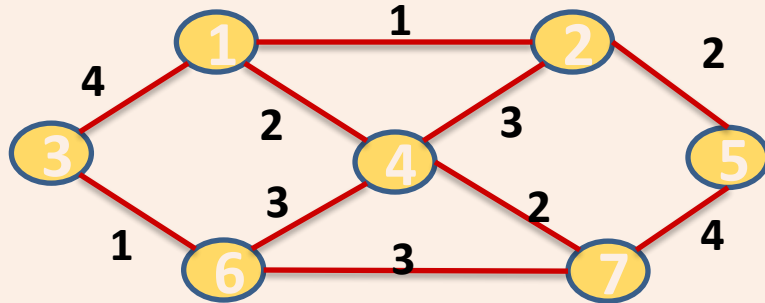
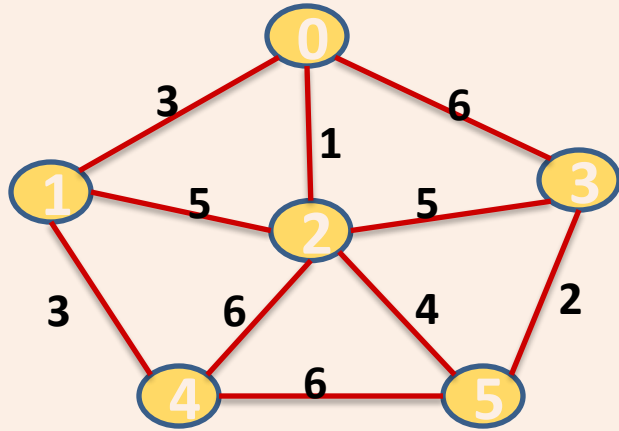
    scanf("%d",&n);

    //entering weighted graph

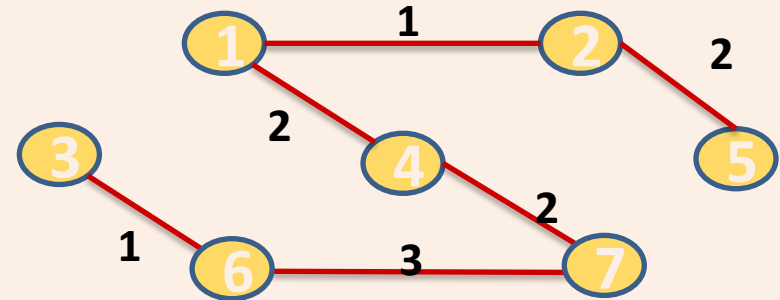
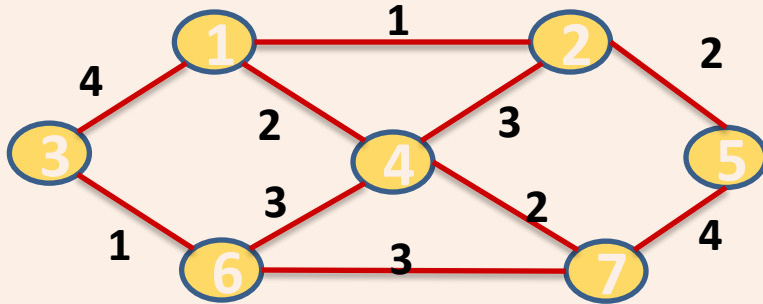
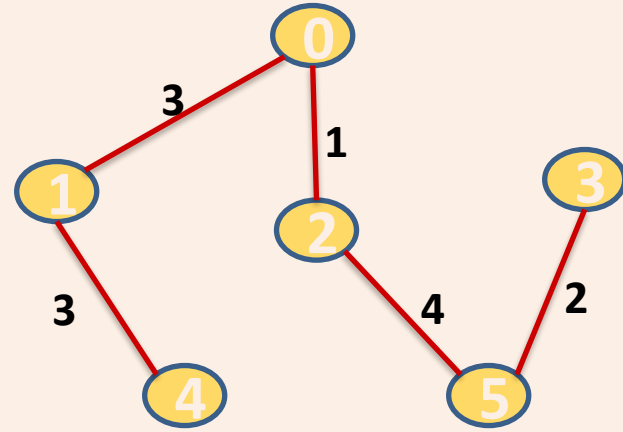
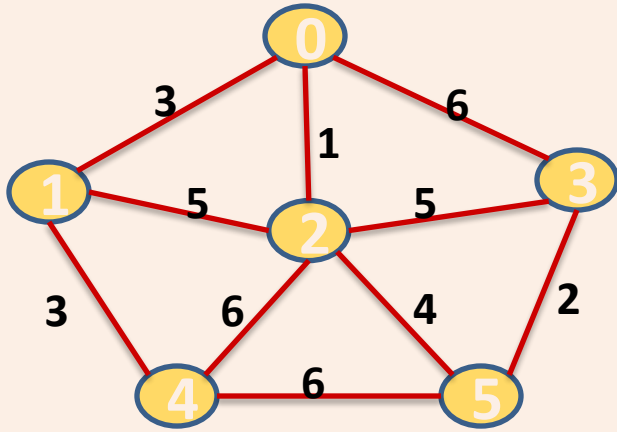
    printf("\nEnter the cost adjacency matrix\n");

    for(i=0;i<n;i++)
    { printf("\n");
        for(j=0;j<n;j++)
        {
```

Construct Minimum Spanning Tree

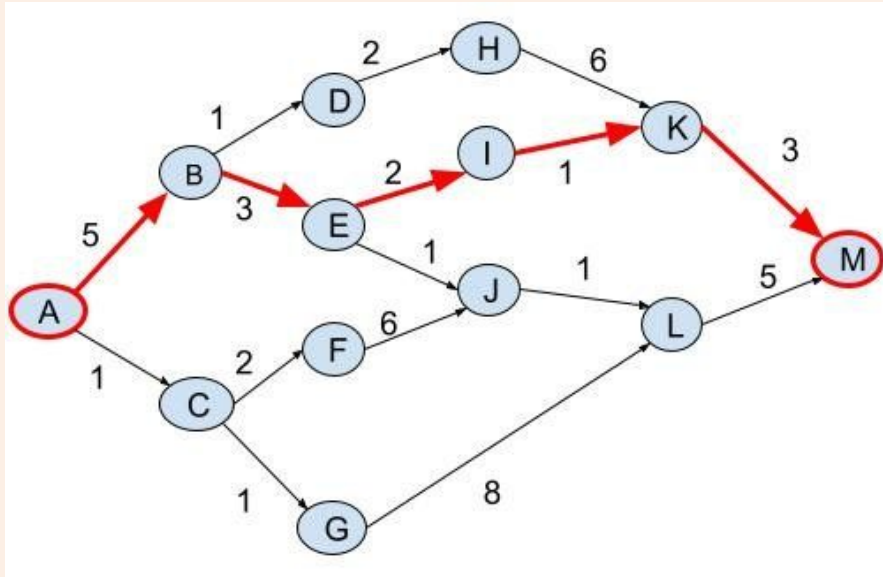


Construct Minimum Spanning Tree



Finding the Shortest Path

- A weighted graph has values (weights) assigned to its edges.
- The length of a path = the sum of the weights of the edges in the path $w(i,j)$ = weight of edge (i,j)

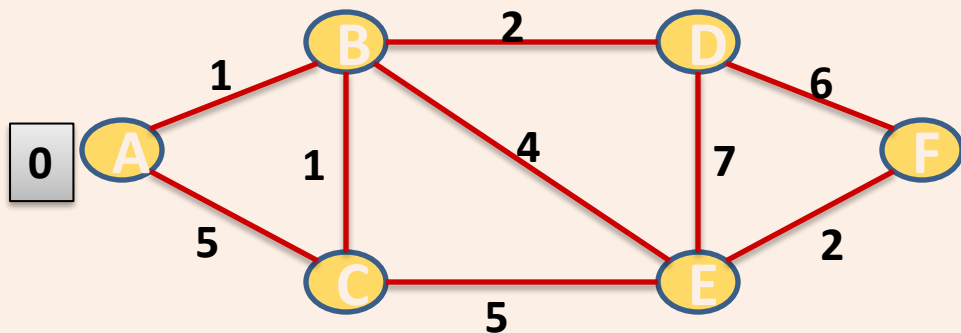


en two vertices is the path
th.



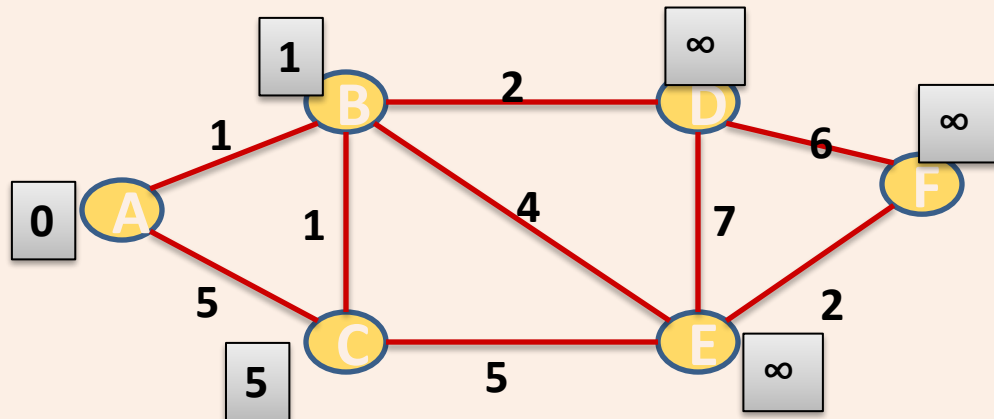
Edsger Wybe Dijkstra 66

Dijkstra's Algorithm



	A	B	C	D	E	F
Distance	0	∞	∞	∞	∞	∞
Visited	0	0	0	0	0	0
Parent	-	-	-	-	-	-

1st Iteration: Select **Vertex A** with minimum distance



	A	B	C	D	E	F
Distance	0	1	5	∞	∞	∞
Visited	1	0	0	0	0	0
Parent	-	A	A	-	-	-

Dijkstra's Algorithm

2nd Iteration: Select **Vertex B** with minimum distance

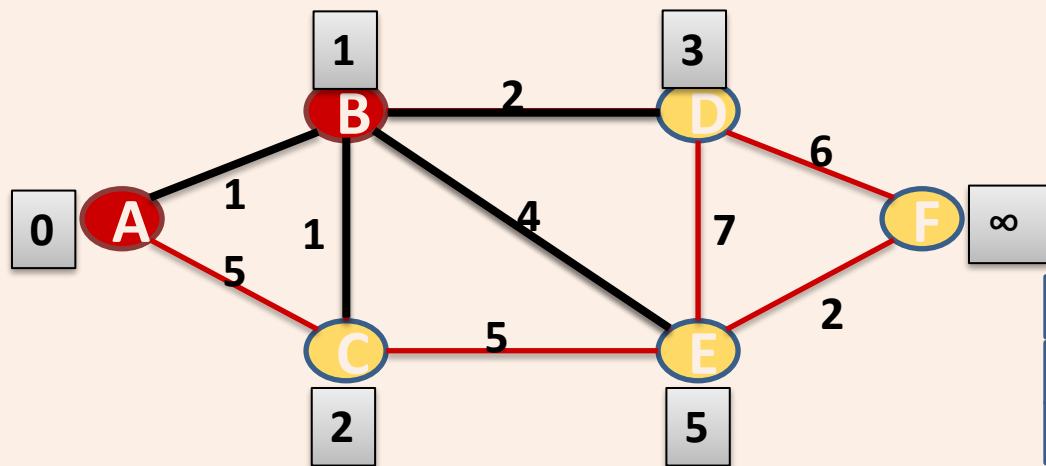
Cost of going to C via B = $\text{dist}[B] + \text{cost}[B][C] = 1 + 1 = 2$

Cost of going to D via B = $\text{dist}[B] + \text{cost}[B][D] = 1 + 2 = 3$

Cost of going to E via B = $\text{dist}[B] + \text{cost}[B][E] = 1 + 4 = 5$

Cost of going to F via B = $\text{dist}[B] + \text{cost}[B][F] = 1 + \infty = \infty$

	A	B	C	D	E	F
Distance	0	1	5	∞	∞	∞
Visited	1	0	0	0	0	0
Parent	-	A	A	-	-	-



	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	0	0	0	0
Parent	-	A	B	B	B	-

Dijkstra's Algorithm

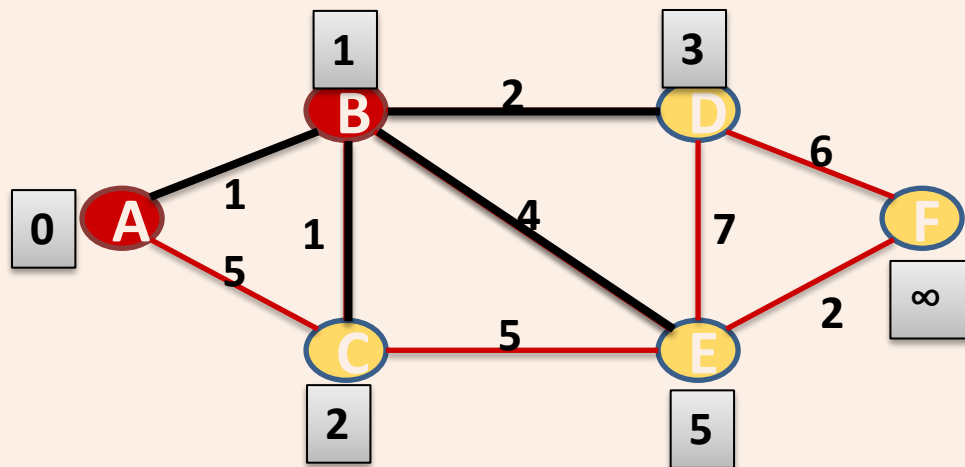
3rd Iteration: Select **Vertex C** via B with minimum distance

Cost of going to D via C = $\text{dist}[C] + \text{cost}[C][D] = 2 + \infty = \infty$

Cost of going to E via C = $\text{dist}[C] + \text{cost}[C][E] = 2 + 5 = 7$

Cost of going to F via C = $\text{dist}[C] + \text{cost}[C][F] = 2 + \infty = \infty$

	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	0	0	0	0
Parent	-	A	B	B	B	-



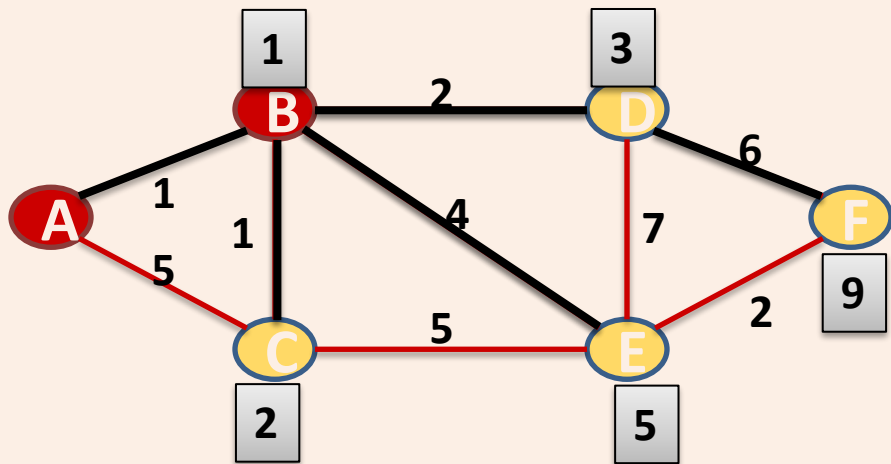
	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	1	0	0	0
Parent	-	A	B	B	B	-

Dijkstra's Algorithm

4th Iteration: Select **Vertex D** via path A - B with minimum distance

Cost of going to E via D = $\text{dist}[D] + \text{cost}[D][E] = 3 + 7 = 10$

Cost of going to F via D = $\text{dist}[D] + \text{cost}[D][F] = 3 + 6 = 9$



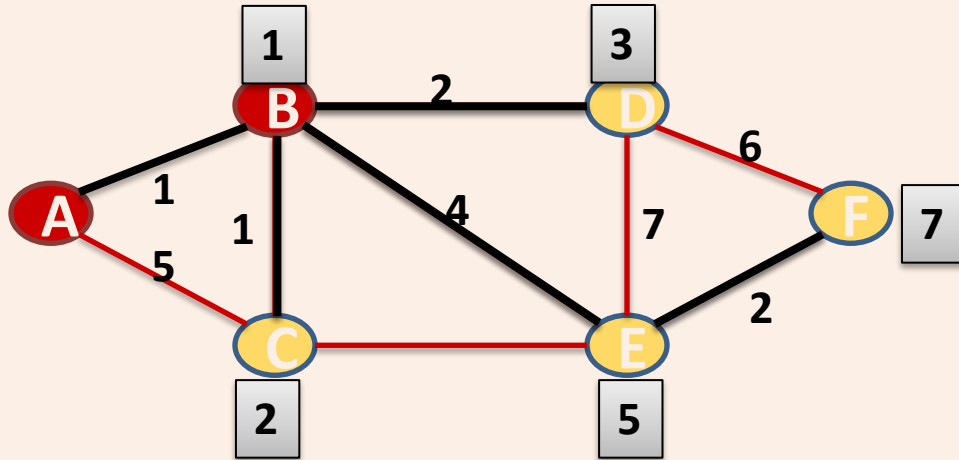
	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	1	0	0	0
Parent	-	A	B	B	B	-

	A	B	C	D	E	F
Distance	0	1	2	3	5	9
Visited	1	1	1	1	0	0
Parent	-	A	B	B	B	D

Dijkstra's Algorithm

4th Iteration: Select **Vertex E** via path A – B – E with minimum distance

Cost of going to F via E = $\text{dist}[E] + \text{cost}[E][F] = 5 + 2 = 7$



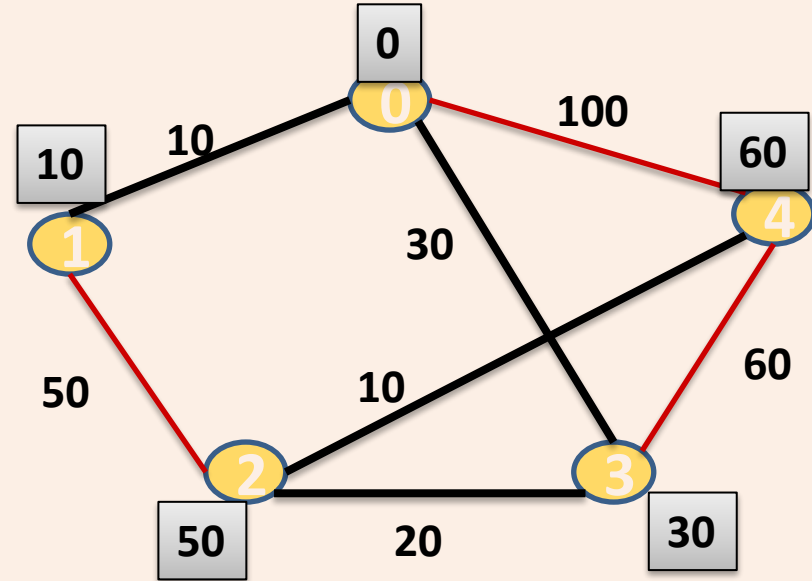
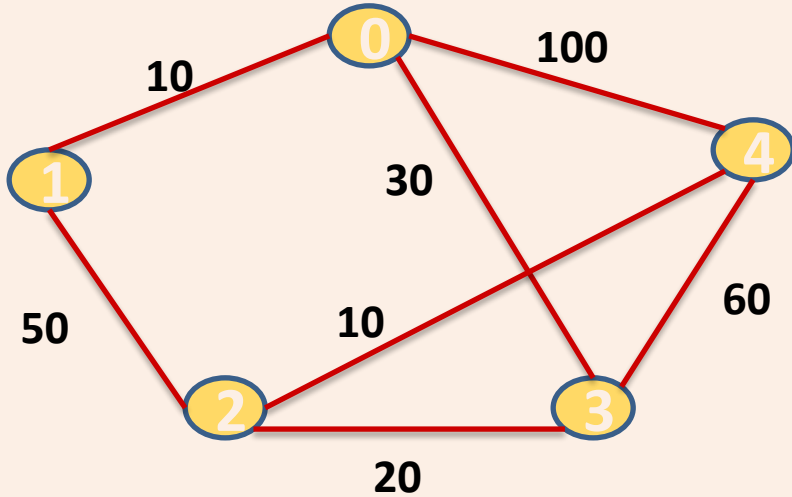
	A	B	C	D	E	F
Distance	0	1	2	3	5	9
Visited	1	1	1	1	0	0
Parent	-	A	B	B	B	D

	A	B	C	D	E	F
Distance	0	1	2	3	5	7
Visited	1	1	1	1	1	0
Parent	-	A	B	B	B	E

Shortest Path from A to F is
A □ B □ E □ F = 7

Dijkstra's Algorithm

Find out shortest path from node 0 to all other nodes using Dijkstra Algorithm





Thank You